



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

INTELLIGENT VACCINE COLD STORAGE MONITORING SYSTEM
INDUSTRY CONTEXT
GIT & DOCKER TECHNICAL ASSIGNMENT

Submitted in the partial fulfilment for the Course of
CSA1016 – SOFTWARE ENGINEERING

to the award of the degree of
BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

Submitted by
SAMEER AHMED G (192421154)

Under the Supervision of
DR ANITA DAVAMANI

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences

Chennai-602105
July 2026

1. PROBLEM ANALYSIS

The Intelligent Vaccine Cold Storage Monitoring System is a software-based healthcare monitoring platform designed to maintain the safety and quality of vaccines stored in refrigerators and cold storage units. Vaccines generally require controlled temperature conditions, and continuous monitoring is necessary because temperature excursions can reduce vaccine effectiveness and result in financial loss and health risks.

Traditional cold storage monitoring may depend on manual temperature recording or independent data loggers. Such approaches can delay the identification of abnormal conditions. The proposed system provides a centralized digital platform that receives sensor readings, monitors vaccine storage conditions, identifies unsafe temperatures, maintains records, and displays the current status through a web dashboard.

The practical implementation developed for this assignment uses a **Flask REST API, HTML/CSS/JavaScript dashboard, and MongoDB database**. Git and GitHub are used for version control and collaboration, while Docker and Docker Compose are used to create a consistent and portable deployment environment.

1.1 Project Objectives

The major objectives are:

- To monitor vaccine storage conditions digitally.
- To receive temperature and humidity readings.
- To identify unsafe temperature conditions.
- To provide a real-time monitoring dashboard.
- To store sensor readings in a database.
- To provide REST API endpoints.
- To display critical alerts.
- To maintain source-code history using Git.
- To containerize the application using Docker.
- To provide reproducible deployment using Docker Compose.

1.2 Functional Requirements

ID	Functional Requirement
FR01	The system shall provide a monitoring dashboard.
FR02	The system shall display vaccine storage units.
FR03	The system shall display temperature readings.
FR04	The system shall display humidity readings.
FR05	The system shall receive sensor readings through an API.
FR06	The system shall identify unsafe temperature conditions.
FR07	The system shall generate critical alerts.
FR08	The system shall store sensor readings in MongoDB.
FR09	The system shall provide health and database status APIs.
FR10	The application shall run inside Docker containers.

1.3 Expected Outcomes

The expected outcome is a working containerized vaccine monitoring application that provides real-time storage-unit information, sensor-data processing, alert identification, and database storage. Git provides controlled source-code management, while Docker provides a reproducible execution environment.

2. PROJECT STRUCTURE DESIGN

The project repository is organized into separate folders for the backend, frontend, testing, and deployment configuration.

vaccine-cold-storage/

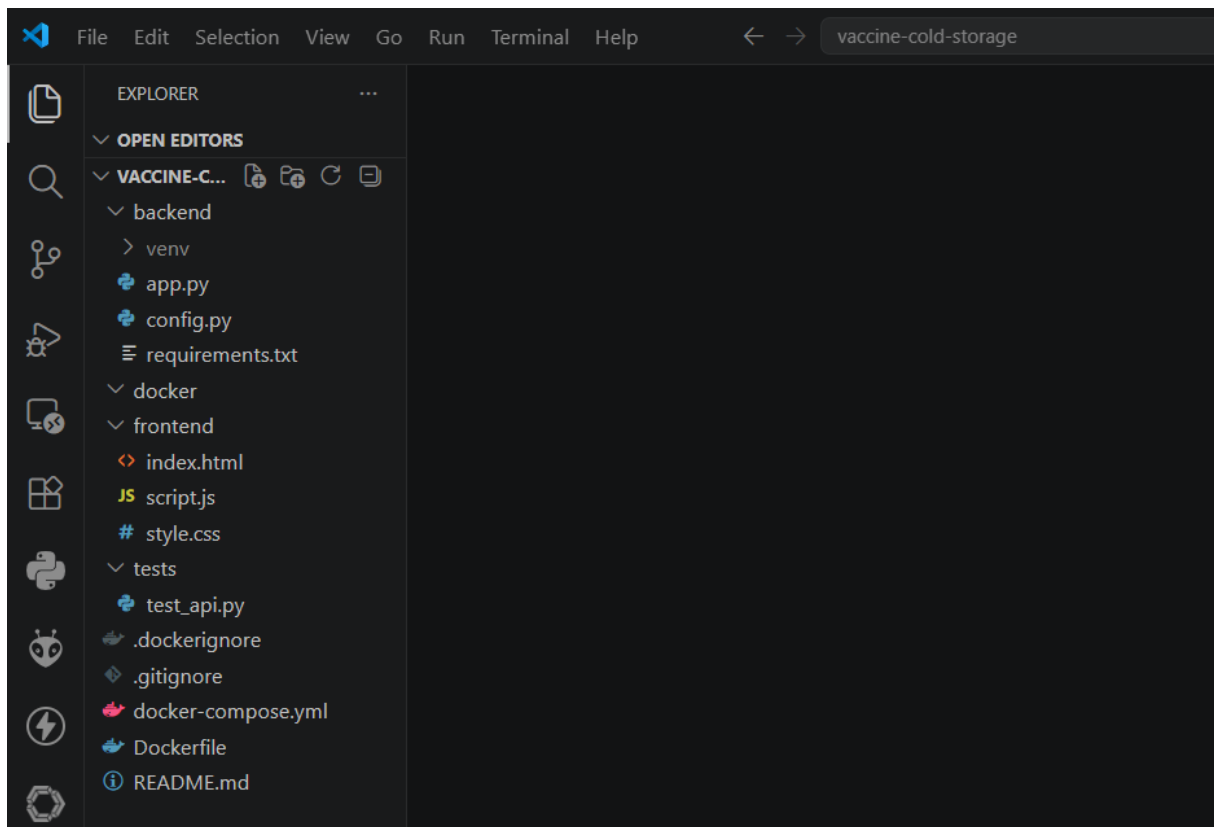
```
|
|
|— backend/
|   |— app.py
|   |— config.py
|   └─ requirements.txt
|
|— frontend/
|   |— index.html
|   |— style.css
|   └─ script.js
|
|— tests/
|   └─ test_api.py
|
|— docker/
|
|— Dockerfile
|— docker-compose.yml
|— .dockerignore
|— .gitignore
└─ README.md
```

The **backend** folder contains the Flask application and configuration files. `app.py` implements the REST API, temperature validation, alert logic, and MongoDB interaction. `requirements.txt` contains the Python dependencies required by the application.

The **frontend** folder contains the monitoring dashboard. `index.html` defines the interface, `style.css` provides the visual design, and `script.js` communicates with the backend API and displays the latest monitoring information.

The **tests** folder is reserved for automated testing. The **Dockerfile** defines how the application image is created, while **docker-compose.yml** defines the backend and MongoDB services, networking, environment variables, and persistent storage.

The **.gitignore** file prevents unnecessary files such as virtual environments and Python cache files from being committed. The **.dockerignore** prevents unnecessary files from being copied into the Docker image. **README.md** provides project documentation.



3. GIT REPOSITORY INITIALIZATION

Git was selected as the version control system because it provides distributed version control, branch management, change tracking, and collaboration capabilities. It allows the development history of the vaccine monitoring system to be maintained systematically.

The repository was initialized using:

`git init`

`git branch -M main`

`git status`

The `git init` command creates a new Git repository in the project directory. The `git branch -M main` command establishes `main` as the primary branch. `git status` displays the current state of tracked and untracked files.

The project files were then staged and committed:

`git add .`

`git commit -m "chore: initialize vaccine monitoring project"`

The first commit establishes the initial version of the project.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Sameer G> cd D:\vaccine-cold-storage
PS D:\vaccine-cold-storage> git init
Initialized empty Git repository in D:\vaccine-cold-storage/.git/
PS D:\vaccine-cold-storage> git branch -M main
PS D:\vaccine-cold-storage> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .dockerignore
        .gitignore
        Dockerfile
        README.md
        backend/
        docker-compose.yml
        frontend/
        tests/

nothing added to commit but untracked files present (use "git add" to track)
PS D:\vaccine-cold-storage>
```

```
PS D:\vaccine-cold-storage> git add .
PS D:\vaccine-cold-storage> git commit -m "chore: initialize vaccine monitoring project"
[main (root-commit) 9effcee] chore: initialize vaccine monitoring project
12 files changed, 417 insertions(+)
create mode 100644 .dockerignore
create mode 100644 .gitignore
create mode 100644 Dockerfile
create mode 100644 README.md
create mode 100644 backend/app.py
create mode 100644 backend/config.py
create mode 100644 backend/requirements.txt
create mode 100644 docker-compose.yml
create mode 100644 frontend/index.html
create mode 100644 frontend/script.js
create mode 100644 frontend/style.css
create mode 100644 tests/test_api.py
PS D:\vaccine-cold-storage> git log --oneline
9effcee (HEAD -> main) chore: initialize vaccine monitoring project
PS D:\vaccine-cold-storage>
```

4. GIT BRANCHING STRATEGY

A feature-based branching strategy was selected for the project. The main branch represents the stable version, while develop is used for integrating completed development work.

The project uses the following branches:

main



The backend API was developed separately using feature/backend-api. The dashboard was developed using feature/frontend-dashboard, while Docker-related implementation was performed using feature/docker.

A feature branch was created using:

```
git checkout -b feature/backend-api
```

After development and testing, the feature was merged into develop:

```
git checkout develop
```

```
git merge feature/backend-api
```

The same workflow was followed for the frontend and Docker features.

This strategy reduces the risk of unstable changes affecting the main branch. It also supports collaborative development because different developers can work on separate features.

```
PS D:\vaccine-cold-storage> git checkout -b develop
Switched to a new branch 'develop'
PS D:\vaccine-cold-storage> git branch
* develop
  main
PS D:\vaccine-cold-storage>
```

```
PS D:\vaccine-cold-storage> git status
On branch develop
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   backend/app.py

no changes added to commit (use "git add" and/or "git commit -a")
PS D:\vaccine-cold-storage> git add backend/app.py
PS D:\vaccine-cold-storage> git commit -m "feat: add system information API"
[develop 59e4d25] feat: add system information API
1 file changed, 8 insertions(+)
PS D:\vaccine-cold-storage> git log --oneline
59e4d25 (HEAD -> develop) feat: add system information API
9effcee (main) chore: initialize vaccine monitoring project
PS D:\vaccine-cold-storage>
```

5. VERSION CONTROL WORKFLOW

The project's Git workflow follows a structured development process:

Create Feature



Create Feature Branch



Develop and Test



`git add`



`git commit`



Merge into develop



Testing



Merge into main



Push to GitHub

The major Git operations performed were:

`git status`

`git add .`

`git commit`

`git branch`

`git checkout`

`git merge`

`git log`

`git push`

`git status` was used to inspect modifications. `git add` staged changes, while `git commit` permanently recorded them in the repository. Branches were created using `git checkout -b`, and completed features were integrated using `git merge`.

The project was synchronized with GitHub using:

```
git remote add origin <repository-url>
```

```
git push -u origin main
```

The synchronization allows the project to be stored remotely and accessed by collaborators.

5.1 Commit History

Meaningful commit messages were used to make the development history understandable.

Example:

feat: add system information API

feat: add dashboard system status

feat: add Docker deployment with MongoDB

chore: initialize vaccine monitoring project

The project follows a Conventional Commits-style approach. feat: represents a new feature, fix: represents a correction, test: represents testing changes, docs: represents documentation, and chore: represents maintenance or configuration work.

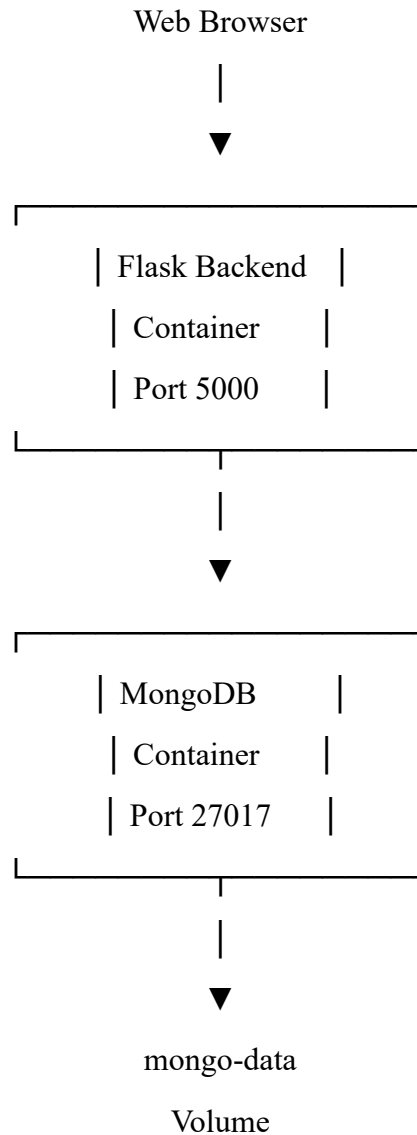
This approach improves maintainability because developers can understand the purpose of individual changes directly from the commit history.

```
PS D:\vaccine-cold-storage> git add .
PS D:\vaccine-cold-storage> git commit -m "feat: add Docker deployment with MongoDB"
[feature/docker f956075] feat: add Docker deployment with MongoDB
5 files changed, 102 insertions(+), 4 deletions(-)
PS D:\vaccine-cold-storage> git log --oneline
f956075 (HEAD -> feature/docker) feat: add Docker deployment with MongoDB
c08c2e6 (feature/frontend-dashboard, develop) feat: add dashboard system status
59e4d25 feat: add system information API
9effcee (main) chore: initialize vaccine monitoring project
PS D:\vaccine-cold-storage> git checkout develop
Switched to branch 'develop'
PS D:\vaccine-cold-storage> git merge feature/docker
Updating c08c2e6..f956075
Fast-forward
 Dockerfile      | 14 ++++++
 backend/app.py  | 48 ++++++
 docker-compose.yml | 42 ++++++
 frontend/index.html | 4 +-
 frontend/script.js | 2 +-
5 files changed, 102 insertions(+), 4 deletions(-)
PS D:\vaccine-cold-storage> git log --oneline --graph --all
* f956075 (HEAD -> develop, feature/docker) feat: add Docker deployment with MongoDB
* c08c2e6 (feature/frontend-dashboard) feat: add dashboard system status
* 59e4d25 feat: add system information API
* 9effcee (main) chore: initialize vaccine monitoring project
PS D:\vaccine-cold-storage> git status
On branch develop
nothing to commit, working tree clean
PS D:\vaccine-cold-storage> git checkout main
Switched to branch 'main'
PS D:\vaccine-cold-storage> git merge develop
Updating 9effcee..f956075
Fast-forward
 Dockerfile      | 14 ++++++
 backend/app.py  | 52 ++++++
 docker-compose.yml | 42 ++++++
 frontend/index.html | 11 ++++++
 frontend/script.js | 18 ++++++
5 files changed, 133 insertions(+), 4 deletions(-)
PS D:\vaccine-cold-storage> git log --oneline --graph --all
* f956075 (HEAD -> main, feature/docker, develop) feat: add Docker deployment with MongoDB
* c08c2e6 (feature/frontend-dashboard) feat: add dashboard system status
* 59e4d25 feat: add system information API
* 9effcee chore: initialize vaccine monitoring project
PS D:\vaccine-cold-storage>
```

6. DOCKER ENVIRONMENT DESIGN

Docker was selected because it provides application isolation, portability, dependency management, and reproducible deployment. Without containerization, differences between development environments can cause dependency and configuration problems.

The application uses two primary containers:



The Flask container contains the application and REST API. The MongoDB container stores sensor readings and application data.

Both containers communicate through a dedicated Docker network called vaccine-network.

7. DOCKERFILE DEVELOPMENT

The Dockerfile defines the environment required to run the Flask application.

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY backend/requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY backend/ ./backend/
```

```
COPY frontend/ ./frontend/
```

```
EXPOSE 5000
```

```
CMD ["python", "backend/app.py"]
```

FROM selects the Python 3.12 slim image as the base environment. WORKDIR establishes /app as the working directory. The COPY instruction transfers the requirements file into the image. RUN installs the required Python packages.

The backend and frontend source code are then copied into the image. EXPOSE 5000 documents the port used by Flask, while CMD specifies the command executed when the container starts.

The Dockerfile provides a reproducible environment because the same image can be built on different systems.

8. DOCKER COMPOSE DESIGN

Docker Compose is used to define and manage the multiple containers required by the application.

services:

backend:

build:

context: .

dockerfile: Dockerfile

container_name: vaccine-backend

ports:

- "5000:5000"

environment:

MONGO_URI: mongodb://mongo:27017/vaccine_db

PORT: 5000

depends_on:

- mongo

networks:

- vaccine-network

mongo:

image: mongo:7

container_name: vaccine-mongodb

ports:

- "27017:27017"

volumes:

- mongo-data:/data/db

networks:

- vaccine-network

networks:

vaccine-network:

driver: bridge

volumes:

mongo-data:

The **backend service** builds the Flask application using the Dockerfile. Port 5000 is mapped between the host machine and the container.

The **MongoDB service** uses the official MongoDB image. Its database files are stored in the mongo-data volume.

The vaccine-network allows the backend to communicate with MongoDB using the service name mongo. The MONGO_URI environment variable therefore uses:

mongodb://mongo:27017/vaccine_db

The depends_on configuration ensures that the MongoDB service is started before the backend service.

The persistent volume prevents database information from being lost when containers are stopped and recreated.

9. CONTAINER DEPLOYMENT AND TESTING

The Docker environment was built using:

`docker compose build`

The command reads the Dockerfile and creates the required application image.

```
Windows PowerShell
#10 28.39 1.7/1.7 MB 88.9 kB/s eta 0:00:00
#10 28.45 Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
#10 28.54 Downloading click-8.4.2-py3-none-any.whl (119 kB)
#10 29.23 Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
#10 32.48 Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
#10 32.58 Downloading Jinja2-3.1.6-py3-none-any.whl (134 kB)
#10 33.78 Downloading MarkupSafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
#10 34.14 Downloading Werkzeug-3.1.8-py3-none-any.whl (226 kB)
#10 36.10 Installing collected packages: MarkupSafe, itsdangerous, dnspython, click, blinker, Werkzeug, pymongo, Jinja2, Flask, flask-cors
#10 37.20 Successfully installed Flask-3.1.1 blinker-1.9.0 click-8.4.2 dnspython-2.8.0 flask-cors-6.0.1 itsdangerous-2.2.0 Jinja2-3.1.6 MarkupSafe-3.0.3 pymongo-4.13.2 Werkzeug-3.1.8
#10 37.20 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
#10 38.30
#10 38.30 [notice] A new release of pip is available: 25.0.1 -> 26.2.1
#10 38.30 [notice] To update, run: pip install --upgrade pip
#10 DONE 38.5s

#11 [5/6] COPY backend/ ./backend/
#11 DONE 0.5s

#12 [6/6] COPY frontend/ ./frontend/
#12 DONE 0.3s

#13 exporting to image
#13 exporting layers
#13 exporting layers 1.8s done
#13 exporting manifest sha256:4022fca0ab156bd249d42e18530ec08d77737ed6ffe0050afd002b30b8d10a6 0.1s done
#13 exporting config sha256:3e25b181d66168066a6d3515bde8409e876b2ff6c6392a02027243c76683d24 0.1s done
#13 exporting config sha256:3e25b181d66168066a6d3515bde8409e876b2ff6c6392a02027243c76683d24 0.1s done
#13 exporting attestation manifest sha256:fd6bb10e2cc79e95ac6ea854f898ef9e642f2e23e0b9946aa76bd9892535fcf 0.2s done
#13 exporting attestation manifest sha256:fd6bb10e2cc79e95ac6ea854f898ef9e642f2e23e0b9946aa76bd9892535fcf 0.2s done
#13 exporting manifest list sha256:380b6123e43ab927162bdd8fba76ce77fd607b834a1cb610cb198c4242b26cce 0.1s done
#13 exporting manifest list sha256:380b6123e43ab927162bdd8fba76ce77fd607b834a1cb610cb198c4242b26cce 0.1s done
#13 naming to docker.io/library/vaccine-cold-storage-backend:latest
#13 naming to docker.io/library/vaccine-cold-storage-backend:latest 0.0s done
#13 unpacking to docker.io/library/vaccine-cold-storage-backend:latest
#13 unpacking to docker.io/library/vaccine-cold-storage-backend:latest 0.8s done
#13 DONE 3.4s

#14 resolving provenance for metadata file
#14 DONE 0.0s
[+] build 1/1
✓ Image vaccine-cold-storage-backend Built

PS D:\vaccine-cold-storage>
```

The application was then started using:

`docker compose up -d`

The running containers were verified using:

`docker compose ps`

The expected deployment contains:

`vaccine-backend`

`vaccine-mongodb`

with the backend exposed through port 5000.

```
PS D:\vaccine-cold-storage> docker compose logs backend
vaccine-backend | * Serving Flask app 'app'
vaccine-backend | * Debug mode: on
vaccine-backend | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
vaccine-backend | * Running on all addresses (0.0.0.0)
vaccine-backend | * Running on http://172.18.0.3:5000
vaccine-backend | Press CTRL+C to quit
vaccine-backend | * Restarting with stat
vaccine-backend | * Debugger is active!
vaccine-backend | * Debugger PIN: 206-217-412
PS D:\vaccine-cold-storage> docker compose logs mongo
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.325+00:00"},"s":"I", "c":"CONTROL", "id":23285, "ctx":"main","msg":"Automatically disabling TLS 1.0, to force-en
vaccine-mongodb | able TLS 1.0 specify --sslDisabledProtocols 'none'"}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.327+00:00"},"s":"I", "c":"NETWORK", "id":4915701, "ctx":"main","msg":"Initialized wire specification","attr":{"spe
vaccine-mongodb | c":{"incomingExternalClient":{"minWireVersion":0,"maxWireVersion":21},"incomingInternalClient":{"minWireVersion":0,"maxWireVersion":21},"outgoing":{"minWireVersion":6,"ma
vaccine-mongodb | xWireVersion":21},"isInternalClient":true}}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.327+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting thread pool","attr":{"poolName":"pa
vaccine-mongodb | rallel execution pool","numThreads":0,"minThreads":0,"maxThreads":128}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.327+00:00"},"s":"I", "c":"NETWORK", "id":4648601, "ctx":"main","msg":"Implicit TCP FastOpen unavailable. If TCP Fa
vaccine-mongodb | stOpen is required, set tcpFastOpenServer, tcpFastOpenClient, and tcpFastOpenQueueSize."}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.328+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting thread pool","attr":{"poolName":"Th
vaccine-mongodb | readPool","numThreads":0,"minThreads":0,"maxThreads":1}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.328+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting thread pool","attr":{"poolName":"Au
vaccine-mongodb | thorizationManager","numThreads":0,"minThreads":0,"maxThreads":1000000000}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.328+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting thread pool","attr":{"poolName":"OC
vaccine-mongodb | SPCache","numThreads":0,"minThreads":0,"maxThreads":8}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting thread pool","attr":{"poolName":"Th
vaccine-mongodb | readPool","numThreads":0,"minThreads":0,"maxThreads":1}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting thread pool","attr":{"poolName":"In
vaccine-mongodb | dexBuildCoordinatorMongod","numThreads":0,"minThreads":0,"maxThreads":1000000000}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"I", "c":"REPL", "id":5123008, "ctx":"main","msg":"Successfully registered PrimaryOnlyService",
vaccine-mongodb | "attr":{"service":"TenantMigrationDonorService","namespace":"config.tenantMigrationDonors"}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"I", "c":"REPL", "id":5123008, "ctx":"main","msg":"Successfully registered PrimaryOnlyService",
vaccine-mongodb | "attr":{"service":"TenantMigrationRecipientService","namespace":"config.tenantMigrationRecipients"}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"W", "c":"NETWORK", "id":11621101, "ctx":"main","msg":"Overriding max connections to honor 'capMemo
vaccine-mongodb | ryConsumptionForPreAuthBuffers' settings","attr":{"limit":99353}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"I", "c":"CONTROL", "id":5945603, "ctx":"main","msg":"Multi threading initialized"}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting thread pool","attr":{"poolName":"Re
vaccine-mongodb | adwriteConcernDefault","numThreads":0,"minThreads":0,"maxThreads":1}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"I", "c":"TENANT_M", "id":7091600, "ctx":"main","msg":"Starting TenantMigrationAccessBlockerRegistr
vaccine-mongodb | y"}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"main","msg":"Starting thread pool","attr":{"poolName":"Te
vaccine-mongodb | nantMigrationBlockerAsyncThreadPool","numThreads":0,"minThreads":0,"maxThreads":4}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"I", "c":"CONTROL", "id":4615611, "ctx":"initandlisten","msg":"MongoDB starting","attr":{"pid":1,"
vaccine-mongodb | port":27017,"dbPath":"/data/db","architecture":"64-bit","host":"f10b4c121299"}}
vaccine-mongodb | {"t":{"$date":"2026-08-09T11:40:50.329+00:00"},"s":"I", "c":"CONTROL", "id":23403, "ctx":"initandlisten","msg":"Build Info","attr":{"buildInfo":{"v
vaccine-mongodb | ersion":"7.0.39","gitVersion":"636e201a5c0732b95e88bb011a72938ce01d290b","opensslVersion":"OpenSSL 3.0.2 15 Mar 2022","modules":[],"allocator":"tcmalloc","environment":{"
```

9.1 Container Log Testing

The backend logs were inspected using:

```
docker compose logs backend
```

The logs confirm that the Flask application started successfully and is listening on the configured port.

9.2 API Testing

The root API was tested using:

```
http://localhost:5000
```

The health endpoint was tested using:

```
http://localhost:5000/api/health
```

A successful response confirms that the backend is operational.

The database connection was tested using:

```
http://localhost:5000/api/db-status
```

The expected result is:

```
{
  "database": "MongoDB",
  "status": "connected"
}
```

9.3 Dashboard Testing

The monitoring dashboard was accessed using:

`http://localhost:5000/dashboard`

The dashboard displays storage units, temperature, humidity, power status, and alert information.

9.4 Sensor Data Testing

A sensor reading was submitted using the API:

Invoke-RestMethod `

-Uri `http://localhost:5000/api/sensor-reading` `

-Method POST `

-ContentType "application/json" `

-Body '{"unit_id":"VACCINE-01","temperature":5.2,"humidity":54}'

The system classified the temperature as safe because it falls within the configured range of 2°C to 8°C.

An abnormal reading was then submitted:

Invoke-RestMethod `

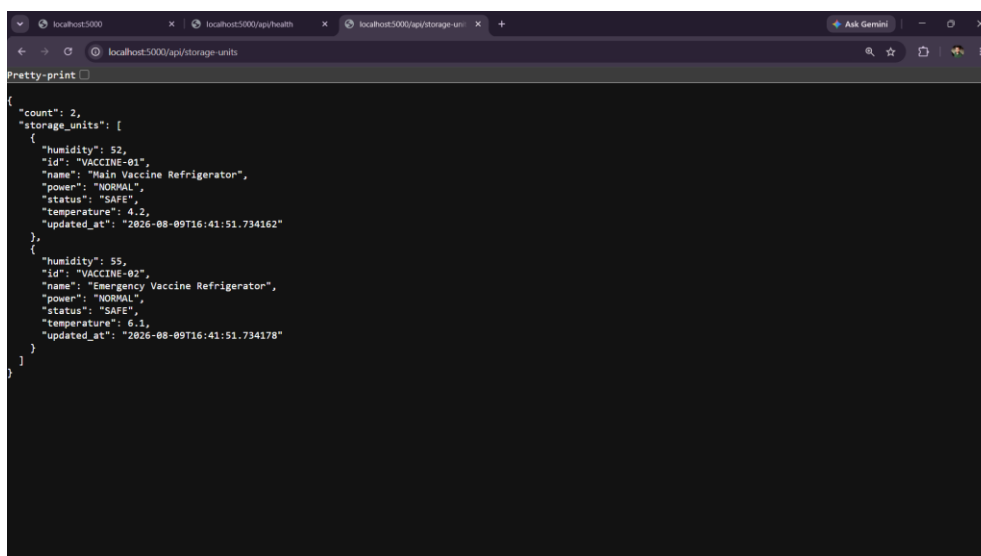
-Uri `http://localhost:5000/api/sensor-reading` `

-Method POST `

-ContentType "application/json" `

-Body '{"unit_id":"VACCINE-01","temperature":10.5,"humidity":58}'

The system classified this reading as CRITICAL, demonstrating the temperature validation and alert mechanism.



9.5 Database Testing

Sensor readings were verified inside MongoDB using:

```
docker exec -it vaccine-mongodb mongosh
```

Then:

```
use vaccine_db
```

```
db.sensor_readings.find().pretty()
```

The stored records demonstrate that sensor information received through the API is successfully persisted in the MongoDB container.

10. BENEFITS OF GIT AND DOCKER

Git provides systematic version management for the project. Developers can create branches, work independently, merge completed features, inspect previous versions, and synchronize changes through GitHub.

Docker complements Git by providing a consistent execution environment. The application and its dependencies are packaged into containers, reducing environment-related problems and simplifying deployment.

The combined use of Git and Docker provides several benefits:

- Better collaboration.
- Complete source-code history.
- Easy rollback.
- Feature isolation.
- Reproducible development environments.
- Portable deployment.
- Dependency isolation.
- Faster application setup.
- Easier testing.
- Improved maintainability.
- Better preparation for CI/CD.
- Support for cloud deployment.

11. CHALLENGES AND IMPROVEMENTS

During implementation, several technical challenges can occur. Git branching can result in merge conflicts when multiple developers modify the same files. Docker may also produce issues related to port conflicts, image building, container networking, environment variables, or database connectivity.

MongoDB connectivity requires particular attention because the hostname inside Docker Compose is different from the hostname used during local development. The application therefore uses an environment variable so that the MongoDB connection can be configured according to the deployment environment.

Another challenge is ensuring database persistence. A Docker volume is used so that MongoDB data remains available after containers are restarted.

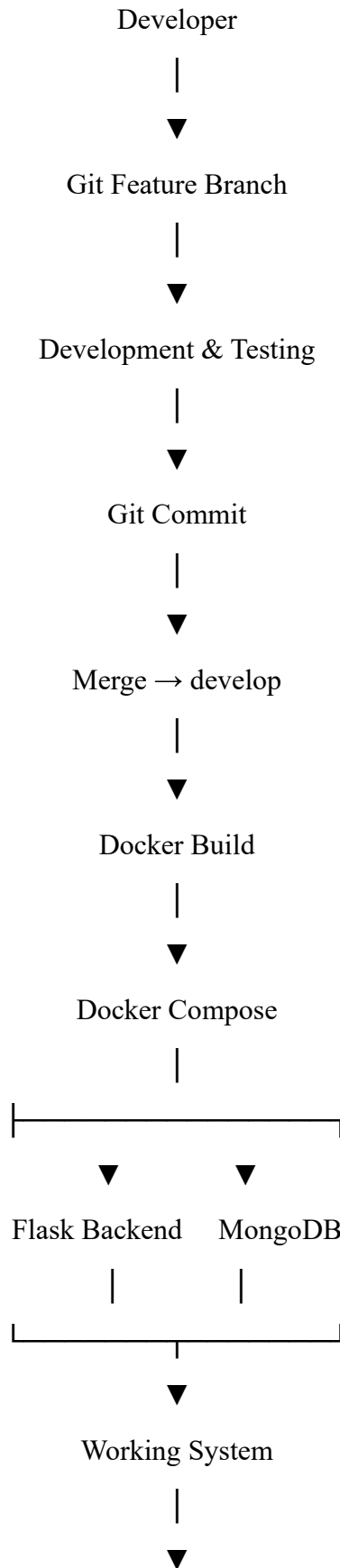
11.1 Future Improvements

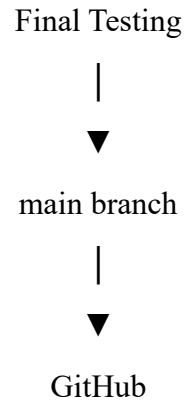
The project can be enhanced with:

- GitHub Actions CI/CD.
- Automated unit and integration testing.
- Docker image security scanning.
- Docker Healthchecks.
- Nginx reverse proxy.
- HTTPS.
- Kubernetes deployment.
- Cloud hosting.
- Automated database backups.
- Real IoT sensors.
- Machine Learning-based equipment failure prediction.
- SMS and email notifications.
- Mobile application.
- Advanced analytics.

12. FINAL GIT AND DOCKER WORKFLOW

The final development workflow can be represented as:





This workflow demonstrates the integration of version control, collaborative development, containerization, deployment, and testing.

13. CONCLUSION

The Intelligent Vaccine Cold Storage Monitoring System was successfully designed as a practical full-stack application and prepared for containerized deployment. Git was used to manage the development lifecycle through repository initialization, branching, meaningful commits, merging, and GitHub synchronization.

Docker was used to create a consistent application environment, while Docker Compose was used to manage the Flask backend and MongoDB database as connected services. The implementation also demonstrates persistent storage through Docker volumes and service communication through a dedicated Docker network.

The final system provides a foundation for real-time vaccine storage monitoring and can be extended with IoT sensors, Machine Learning, cloud deployment, CI/CD, and mobile notifications. The practical integration of Git and Docker demonstrates how modern software engineering and DevOps practices can improve collaboration, portability, maintainability, and deployment reliability.

14. REFERENCES

- [1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill, 2020.
- [3] Git, “Git Documentation.” [Git Documentation](#)
- [4] GitHub, “GitHub Documentation.” [GitHub Documentation](#)
- [5] Docker, “Docker Documentation.” [Docker Documentation](#)
- [6] Docker, “Docker Compose Documentation.” [Docker Compose Documentation](#)
- [7] A. Merkel, “Docker: Lightweight Linux Containers for Consistent Development and Deployment,” *Linux Journal*, no. 239, 2014.
- [8] M. Fowler, “Continuous Integration,” *Martin Fowler*. [Continuous Integration](#)
- [9] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [10] World Health Organization, *Temperature Sensitivity of Vaccines*. Geneva, Switzerland: WHO, 2006.